



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ



ДОКУМЕНТАЦИЯ

Дисциплина: Софтуерни шаблони

Задание: Да се създаде програма за редактиране на текст с опции за добавяне, изтриване, маркиране и копиране на текст на дадена позиция. Демонстрирайте работа със State pattern и Command pattern.

Допълнителен софтуерен шаблон: Memento pattern

Проекта изготвил:

Боян Зарев

фак. номер: 123222004

група: 42

факултет: ФКСТ

специалност: КСИ

курс: IV

e-mail: bzarev@tu-sofia.bg

София, 2025

Съдържание

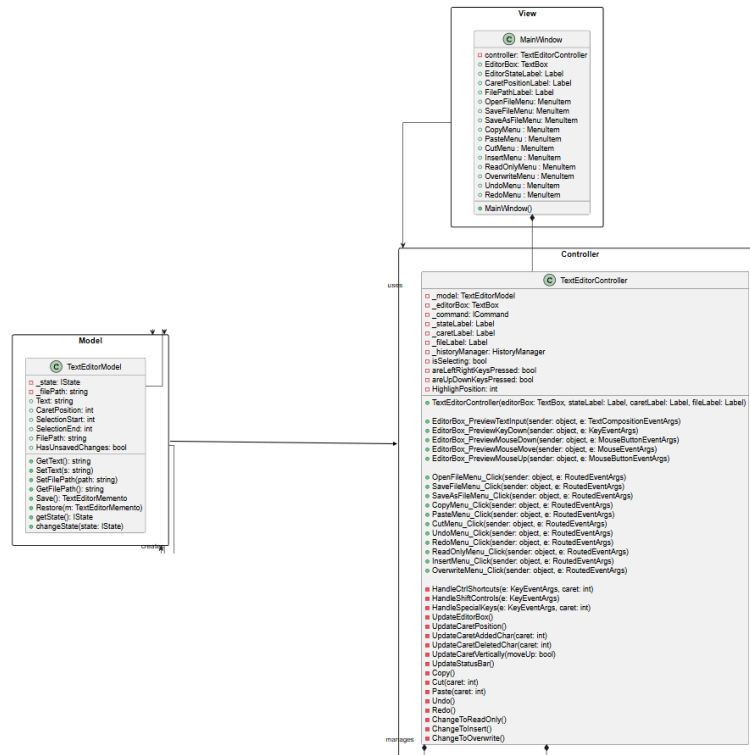
Въведение	1
1. MVC архитектура.....	1
2. Използвани софтуерни шаблони в приложението	2
2.1. State pattern	2
2.2. Command pattern.....	5
2.3. Memento pattern	8
3. UML диаграма	9
4. Описание на потребителския интерфейс и начина на работа с приложението	10
4.1. Контроли	11
5. Стъпки за стартиране.....	12

Въведение

Настоящият проект представлява текстови редактор, разработен на езика C# с помощта на WPF, който предоставя възможност за създаване, редактиране, запазване и отваряне на текстови файлове с разширение .txt. Приложението е изградено върху MVC архитектурен модел (Model–View–Controller), който осигурява ясно разделение между логиката на данните, визуализацията и управлението на взаимодействието с потребителя. В процеса на реализация са използвани и няколко класически шаблона за проектиране: State, Command, Memento. State шаблонът управлява различните състояния на редактора (ReadOnly, Insert, Overwrite, Highlight). Command шаблонът служи за капсулиране на действията на потребителя като отделни команди (въвеждане на текст, изтриване, копиране, поставяне и др.). Memento шаблонът осигурява съхраняването и възстановяването на предишни състояния на текста.

1. MVC архитектура

MVC (Model-View-Controller) е архитектурен модел, който разделя приложението на три основни компонента - Model (Модел), View (Изглед) и Controller (Контролер). Тази структура има за цел да осигури по-добра организация на кода, лесна поддръжка и ясно разделение на отговорностите между отделните части на системата. Моделът (Model) представлява логическото ядро на приложението. Той отговаря за управлението на данните, бизнес логиката и правилата на системата. Моделът съдържа структурата и поведението на данните, като не зависи от начина, по който те се визуализират. В този проект тази функционалност изпълнява TextEditorModel. Той съдържа полета за позицията на курсора, въведения текст, маркираният текст, крайните позиции на маркирания текст и локацията на отворения файл. Изгледът (View) е визуалната част на приложението, която се грижи за представянето на информацията пред потребителя. Той получава данни от модела и ги показва в подходящ формат. Изгледът не съдържа логика за обработка на данните, а само за тяхното изобразяване. Изгледът в случая е MainWindow. Той съдържа меню с команди, поле за въвеждане на текст и текстово поле предназначено за показване на позицията на курсора, състоянието на текстовия редактор и локацията на отворения файл. Контролерът (Controller) служи като посредник между изгледа и модела. Той приема входните действия на потребителя (например натискания на клавиши, бутони или команди), обработва ги и предприема съответните действия върху модела. След промяна в данните, контролерът уведомява изгледа да се актуализира. В проекта контролерът е TextEditorController. Той съдържа функции обработващи събитията и допълнителните функции за промяна на модела и уведомяване на изгледа за промените на модела. Понеже основната идея на този проект е демонстрация на работа на Command, State, Memento софтуерните шаблони, архитектурата MVC няма да бъде детайлно описана. На фигура 1 е показана диаграмата на MVC.



(Фиг. 1) Диаграма на MVC

2. Използвани софтуерни шаблони в приложението

2.1. State pattern

State е поведенчески шаблон, който позволява на обект да променя поведението си динамично в зависимост от своето вътрешно състояние. Шаблонът има за цел да избегне използването на множество условни оператори (if, switch) за проверка. Вместо обектът сам да решава какво да направи във всеки момент, той поддържа препратка към обект, който представя текущото му състояние, което определя поведението му. При промяна на състоянието се заменя този обект без да се променя основната логика на класа. Основните компоненти на този шаблон са:

- Context (Контекст) – класът, който поддържа текущото състояние и предоставя интерфейс за външни операции.
- State (Интерфейс на състоянието) – дефинира общ интерфейс за всички възможни състояния.
- Concrete States (Конкретни състояния) – реализират конкретното поведение за всяко състояние. Всяко състояние може да предизвика промяна към друго състояние при определени условия.

Контекстът в случая е `TextEditorModel`. Той има следните полета:

- `Text: string` – съдържанието на текстовия редактор)
- `CaretPosition: int` (позицията на курсора)
- `SelectionStart: int` (начална позиция на маркирания текст)
- `SelectionEnd: int` (крайна позиция на маркирания текст)
- `HasSelection: bool` (булева променлива показваща съществува ли маркиран текст или не)
- `FilePath: string` (локацията на отворения файл)
- `IState` (състояние на текстовия редактор)
- `Type(text: String)`
- `Delete()`
- `Copy()`
- `Paste()`
- `Cut()`
- `Save()` (запазва копия на текущия модел в Memento шаблона)
- `Restore(memento: TextEditorMemento)` (връща предишното състояние на модела от Memento шаблона)

Интерфейс на състоянието `IState` дефинира следните функции:

- `Type(text: String)`
- `Delete()`
- `Copy()`
- `Paste()`
- `Cut()`

Какво правят тези функции зависи от конкретното състояние. Конкретните състояния имплементират интерфейса и имат референция към контекста. Те са следните:

1. Insert State

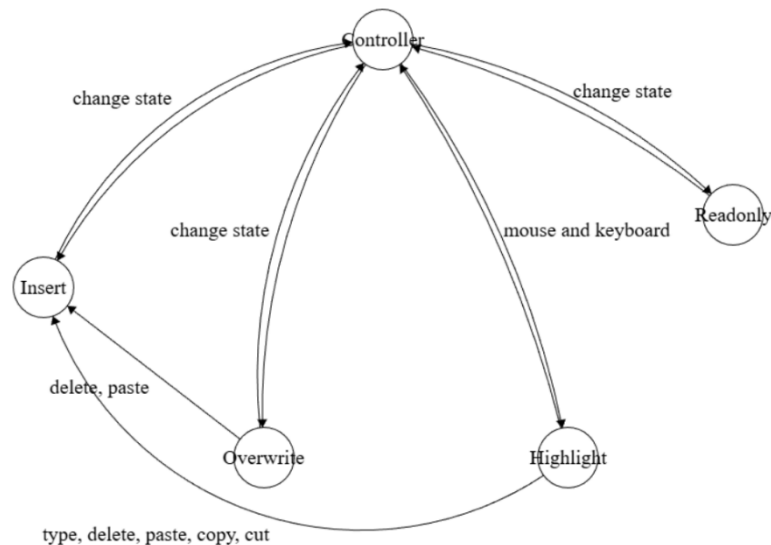
- `Type(text: string)` – вмъква текст в позицията на курсора
- `Delete()` – изтрива един `char` на позицията на курсора
- `Copy()` – не прави нищо
- `Paste()` – вмъква копираният текст на позицията на курсора
- `Cut()` – не прави нищо

2. Overwrite state

- `Type(text: string)` – презаписва текста на позицията на курсора
- `Delete()` – изтрива един `char` на позицията на курсора и сменя състоянието в Insert State

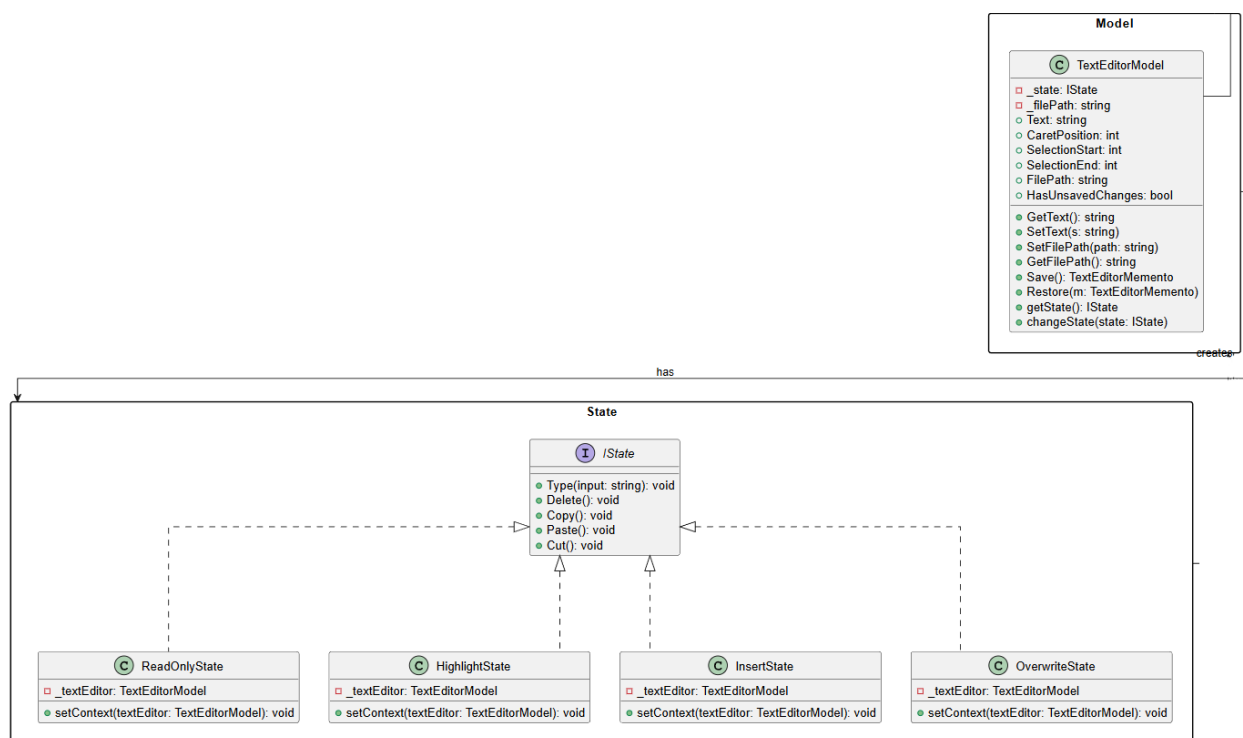
- Copy() – не прави нищо
 - Paste() – вмъква копираният текст на позицията на курсора и сменя състоянието в Insert State
 - Cut() – не прави нищо
3. ReadOnly State
- Type(text: string) – игнорира събитията на клавиатурата
 - Delete() – игнорира събитията на клавиатурата
 - Copy() – не прави нищо
 - Paste() – не прави нищо
 - Cut() – не прави нищо
4. Highlight State
- Type(text: string) – изтрива маркираният текст, слага въведения текст на позицията на курсора и сменя състоянието в Insert State
 - Delete() – изтрива маркираният текст и сменя състоянието в Insert State
 - Copy() – копира маркираният текст в clipboard-a
 - Paste() – изтрива маркирания текст, на неговото място слага копиран текст от clipboard-a и сменя състоянието в Insert State
 - Cut() – изпълнява Copy() след това Delete() и сменя състоянието в Insert State

Автоматът на състоянията е показан на фигура 2.



(Фиг. 2) Автомат на състоянията в текстовия редактор

На фигура 3 е представена UML диаграмата на State софтуерния шаблон в конкретната имплементация.



(Фиг. 3) Диаграма на State софтуерен шаблон

2.2. Command pattern

Command е поведенчески шаблон, който цели да капсулира дадено действие или операция като обект. Това позволява отделянето на логиката за извършване на дадена команда от обекта, който я извиква. С други думи, шаблонът превръща заявките в обекти, което прави възможно лесното им съхранение, поддръждане, отмяна (undo) или повторно изпълнение (redo). Вместо клиентът директно да извиква методи на даден обект, той създава Command обект, който съдържа цялата необходима информация за изпълнението на действието: кой метод да се извика, върху кой обект и с какви параметри. След това този обект се подава на Invoker (например контролер или мениджър на команди), който решава кога и дали да го изпълни. Основните компоненти на този шаблон са:

- Command (Интерфейс или абстрактен клас) – дефинира методите execute(), undo() и redo(), който се имплементира от конкретните команди
- ConcreteCommand (Конкретна команда) – реализира интерфейса Command и съдържа връзка към Receiver. Определя как точно да се изпълни действието
- Receiver (Получател) – обектът, който знае как реално да изпълни операцията
- Invoker (Извикващ) – обектът, който стартира командата, без да знае подробности за нейното изпълнение
- Client (Клиент) – създава конкретните команди и задава техните параметри

TextEditorModel е Reciever, защото той има функционалностите необходими за изпълнението на командите. Основните блокове на командите са методите Type(), Copy(), Paste(), Cut(). Поведението на тези методи зависи от състоянието на модела (Insert, Overwrite, ReadOnly, Highlight). TextEditorModel е описан в подзаглавието „2.1. State pattern“.

Контролерът е Invoker, защото той изпълнява командите. Параметрите на командите се подават от потребителя чрез входа на клавиатурата и мишката или чрез елементите на графичния интерфейс.

Командният интерфейс ICommand дефинира методите execute(), undo() и redo(). Методите undo() и redo() връщат назад или напред състоянията на приложението. Те използват Hisotry Manager от шаблона Memento, който ще бъде описан в следващото подзаглавие. Изпълнението на метода execute(), зависи от конкретната команда. Конкретните команди са следните:

1. AddTextCommand

Като параметър взима текст от тип string, и моделът ще запише текста по начин, зависещ от неговото състояние. Контролерът стартира командата при натиск на клавиш.

2. CopyCommand

Копира маркираният текст в Clipboard-a. Това дали текстът ще се копира или не пак зависи от състоянието на модела. Контролерът стартира командата при натиск на Ctrl+C клавишите или с бутона Copy от менюто намиращо се на върха на прозореца в секция Edit.

3. DeleteCommand

Изтрива един char или изтрива маркирания текст. Контролерът стартира командата при натиск на клавишите Backspace или Delete.

4. OpenCommand

Отваря прозорец OpenFileDialog, където потребителят избира кой файл с .txt разширение да отвори. Контролерът стартира командата при избор на опцията Open от менюто в секция File.

5. PasteCommand

Записва копираният текст от Clipboard-a. Това дали текстът ще се напише или не, зависи от състоянието на модела. Контролерът стартира командата при натиск на Ctrl+V клавишите или с бутона Paste от менюто намиращо се на върха на прозореца в секция Edit.

6. SaveCommand

Записва направените промени във файл. Като параметър взима булева променлива, която детерминира дали при изпълнението на командата ще се отвори прозорец FileSaveDialog или не. В случай ако параметърът е false, тогава по подразбиране FileSaveDialog няма да се отвори, с изключение ако първоначално не е отворен никакъв файл. В случая ако параметърът е True, FileSaveDialog ще се отвори във всеки случай. Контролерът извиква командата параметъра със стойност false при избор на Save от менюто File. Контролерът извиква командата с параметър със стойност true при избор на Save As от менюто File.

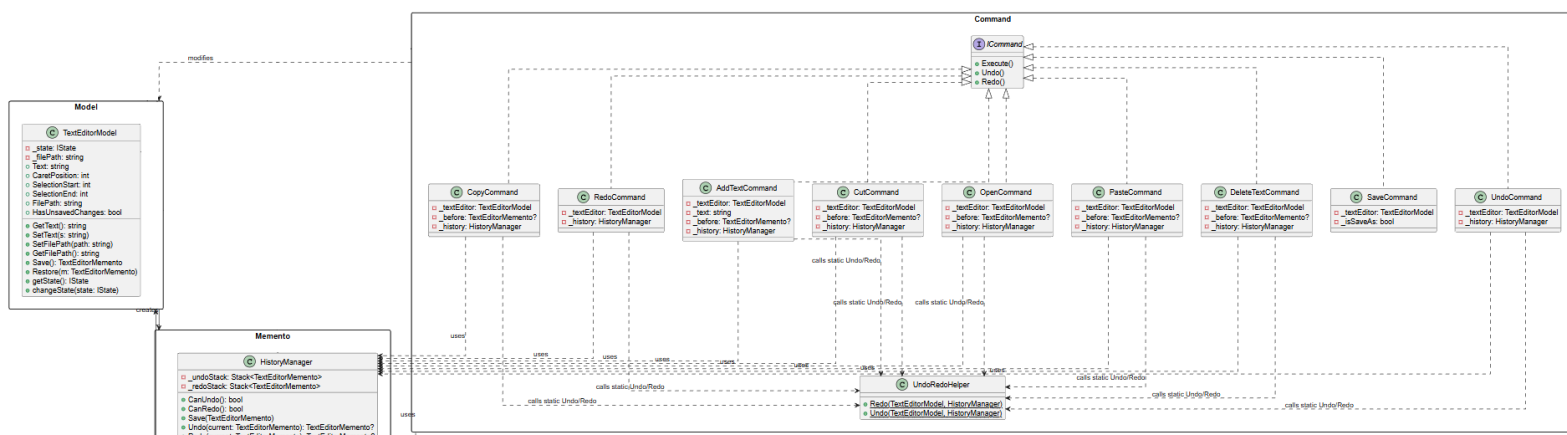
7. UndoCommand

Тази команда връща едно състояние на приложението назад. Въпреки че всички команди имат методите undo() и redo(), командата е направена с цел на осъществяване на по-голяма флексибилност и възможност за добавяне на нови функционалности без промяна на кода на другите команди. Контролерът стартира командата с натиск на Ctrl+Z клавишите или при избор на Undo от менюто в секция Controls.

8. RedoCommand

Тази команда връща едно състояние на приложението напред. Въпреки че всички команди имат методите undo() и redo(), командата е направена с цел на осъществяване на по-голяма флексибилност и възможност за добавяне на нови функционалности без промяна на кода на другите команди. Контролерът стартира командата с натиск на Ctrl+Y клавишите или при избор на Redo от менюто в секция Controls.

На фигура 4 е представена UML диаграмата на Command софтуерния шаблон в конкретната имплементация.



(Фиг. 4) Диаграма на Command софтуерен шаблон

2.3. Memento pattern

Memento е поведенчески шаблон на проектиране, който позволява да се запазва и възстановява предишно състояние на обект, без да се нарушава неговата капсулация. С други думи, той предоставя механизъм за undo или възстановяване на състояние, без външните обекти да имат достъп до вътрешните детайли на този обект. Шаблонът разделя отговорностите между 3 участника:

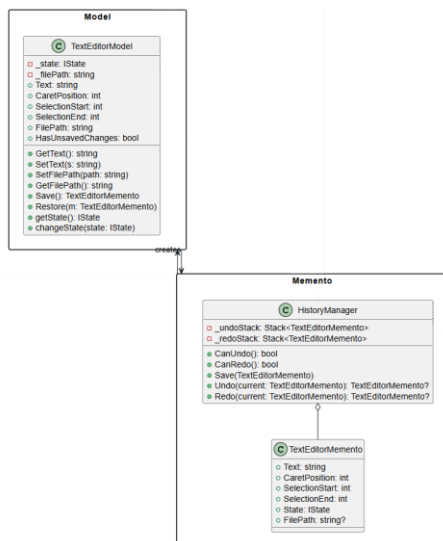
- Originator (Източник) – обектът, чието състояние трябва да бъде запазено или възстановено. Той създава Memento, което съдържа моментна снимка на вътрешното му състояние.
- Memento – обект, който съхранява състоянието на Originator в даден момент от време. Той не позволява директен достъп до вътрешните си данни, освен на самия Originator.
- Caretaker – отговаря за съхранението на Memento обектите, но не ги променя и не разглежда съдържанието им.

TextEditorModel е Originator. Той създава нов Memento обект със метода save(), който съхранява състоянието на модела в определен момент. С метода restore(memento: TextEditorMemento) моделът си връща състоянието на полетата от този запазен snapshot подаден като параметър към метода.

TextEditorMemento е Memento участника. Той съхранява полетата на TextEditorModel.

HistoryManager е Caretaker-а. Той има Undo и Redo стек, където съхранява състоянията на модела. Методите Undo() и Redo() в класа HistoryManager реализират механизма за връщане и повторение на състоянията на модела, като използват принципите на Memento шаблона. Методът Undo(current : TextEditorMemento) се използва, когато потребителят иска да върне последното извършено действие. Той проверява дали има налични състояния за връщане в стека за Undo. Ако има, извлича последния запазен Memento от този стек, добавя текущото състояние на модела в стека за Redo и връща извлечения Memento, който представлява предишното състояние на редактора. По този начин моделът може да възстанови съдържанието си до момента преди последната промяна. Методът Redo(current : TextEditorMemento) изпълнява обратната операция. Когато потребителят иска да повтори действие, което преди това е било отменено, методът проверява дали има налични състояния в стека за Redo. Ако има, извлича следващия Memento от този стек, запазва текущото състояние на модела в стека за Undo и връща извлечения Memento, който съдържа състоянието, което трябва да бъде възстановено. Двата метода се извикват от изтомените методи в командите описани в подзаглавието „2.2. Command pattern“.

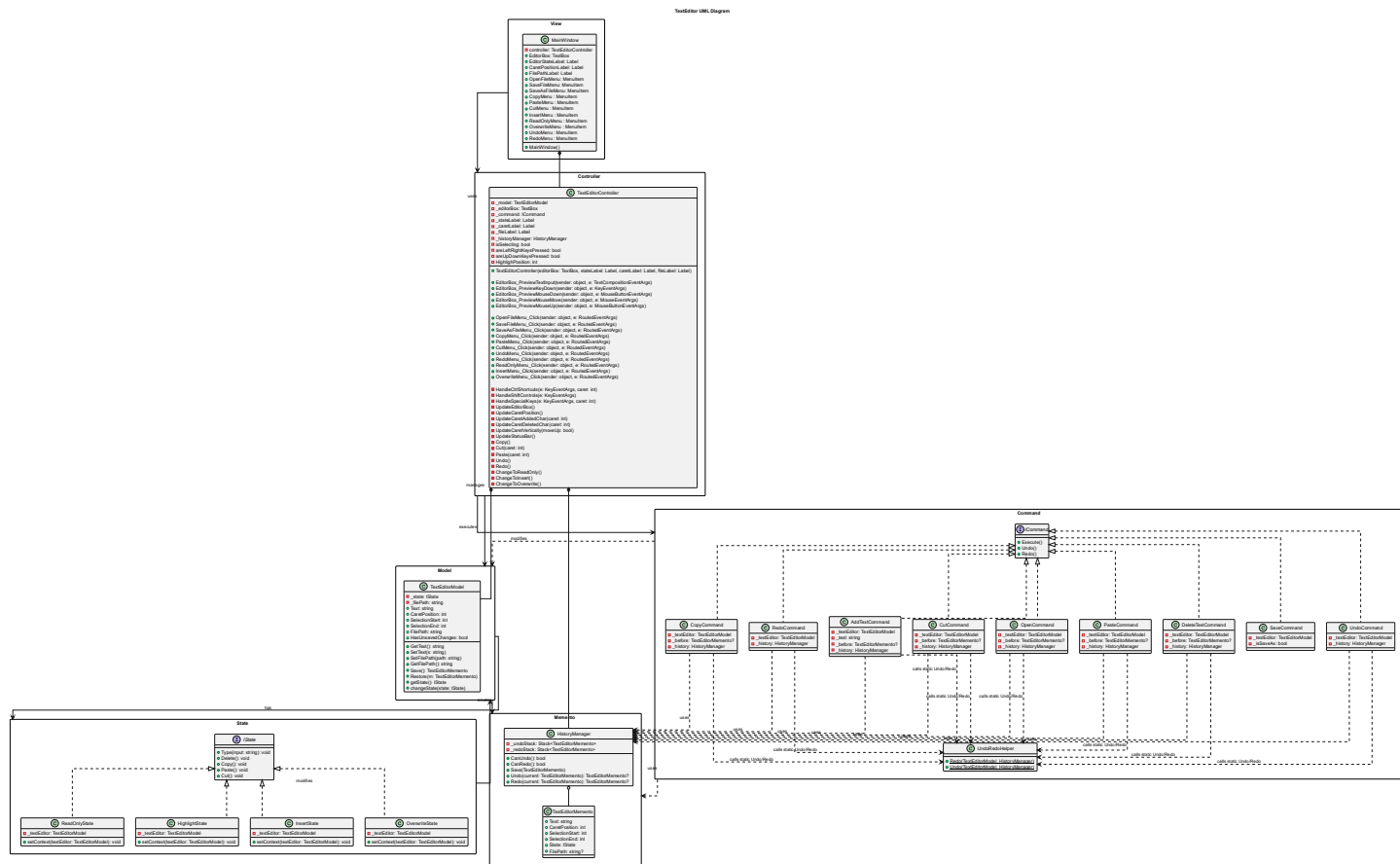
На фигура 5 е представена UML диаграмата на Memento софтуерния шаблон в конкретната имплементация.



(Фиг. 5) Диаграма на Мемето софтуерен шаблон

3. UML диаграма

Цялостната UML диаграма е представена на фигура 6.



(Фиг. 6) UML диаграма

4. Описание на потребителския интерфейс и начина на работа на приложението

Фигура 7 показва как изглежда графичния интерфейс на приложението.



(Фиг. 7) Screenshot от приложението

На фигура 8 са означени частите на графичния интерфейс. С 1 е означено менюто. Менюто има 4 секции: File, където потребителят може да отвори, запише или създаде файл; Edit, където потребителят може да извика командите Copy, Paste и Cut; State, където потребителят може да промени състоянието на текстовия редактор в Insert, Overwrite или ReadOnly състояние; и Controls, където потребителят може да извика Undo и Redo командите. На фигурата с 2 е означено полето, където потребителят въвежда или редактира текст. Това поле е основният интерактивен елемент на приложението. С номера 3 е означено поле, което дава информация за състоянието на текстовия редактор (Insert, Overwrite, ReadOnly, Highlight), позицията на курсора и локацията на отворения файл. В случай ако не е отворен файл, локацията изписва "Untitled".



(Фиг. 8) Screenshot от приложението с означени полета

4.1. Контроли

- Меню

1. File
 - 1.1. Open – отваря файл
 - 1.2. Save – запазва промените ако съществува отворен файл, ако не отваря FileOpenDialog
 - 1.3. Save As – отваря FileOpenDialog
2. Edit
 - 2.1. Copy – слага маркирания текст в Clipboard-a
 - 2.2. Paste – записва текста от Clipboard-a
 - 2.3. Cut – стартира Copy командата и изтрива маркираният текст
3. State
 - 3.1. ReadOnly – променя състоянието в ReadOnly състояние
 - 3.2. Insert – променя състоянието в Insert състояние
 - 3.3. Overwrite – променя състоянието в Overwrite състояние
4. Controls
 - 4.1. Undo – стартира Undo командата
 - 4.2. Redo – стартира Redo командата

- Текстово поле

1. Въвеждане, изтриване, копиране, маркиране на текст
2. Местене на курсора с мишката или с Arrow Keys (Up, Down, Left, Right)
3. Маркиране на текст
 - 3.1. Маркиране с курсора: При събитието на натискане на мишката се определя началната позиция на маркирания текст, при събитието на пускане се определя крайната позиция на маркирания текст. При маркиране, състоянието се сменя в Highlight състояние.
 - 3.2. Маркиране с Shift + Arrow Keys: Когато клавишът Shift натиснат, потребителят може да мести курсора, маркирайки текст. При маркиране състоянието се сменя в Highlight състояние.

- Клавишни комбинации

1. Ctrl + C – Copy
2. Ctrl + V – Paste
3. Ctrl + X – Cut
4. Ctrl + S – Save
5. Ctrl + Z – Undo
6. Ctrl + Y – Redo
7. Ctrl + I – промяна на състоянието в Insert състояние
8. Ctrl + O – промяна на състоянието в Overwrite състояние
9. Ctrl + R – промяна на състоянието в ReadOnly състояние

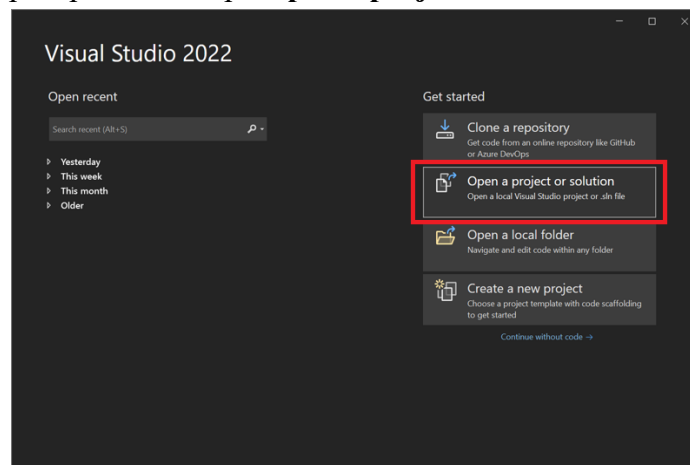
5. Стъпки за стартиране

1. Клониране на репоситорията

```
git clone https://github.com/boce1/TextEditor.git
```

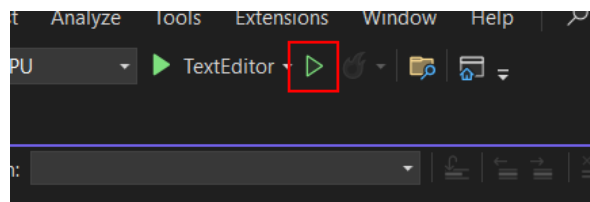
2. Отваряне на проекта с Visual Studio

В началния прозорец се избира **Open a project or solution.**



(Фиг. 9) Screenshot от началния прозорец на Visual Studio

3. Стартиране на програмата с натиск на бутона **Start Without debugging** или с натиск на клавишите **Ctrl + F5**



(Фиг. 10) Бутон Start Without Debugging